MANUAL TÉCNICO DE INSTALACIÓN Y DESPLIEGUE DEL ERP GECO

1. INTRODUCCIÓN

El presente manual técnico describe los procedimientos necesarios para la instalación y despliegue del ERP Geco reingenierizado, una solución empresarial basada en una arquitectura de microservicios con frontend en Angular y backend en Spring Boot.

El sistema está compuesto por los siguientes componentes principales:

Componente	Tecnología	Puerto por defecto	Descripción
API Gateway	Spring Cloud Gateway	8080	Punto único de entrada, enruta peticiones a los microservicios
Eureka Discovery	Spring Cloud	8761	Descubrimiento de servicios
Microservicio Seguridades	Spring Boot	8081	Autenticación JWT, gestión de usuarios y permisos
Microservicio Personas	Spring Boot	8082	Gestión de personas, identificaciones y contactos
Microservicio Productos	Spring Boot	8083	Parametrización de productos, itinerarios y tarifas
Microservicio Proveedores	Spring Boot	8084	Gestión de proveedores e insumos
Microservicio Reservas	Spring Boot	8085	Gestión integral de reservas
Frontend	Angular 21	4200	Interfaz de usuario (SPA)
Base de datos	MySQL 5.7	3306	Persistencia de datos

2. REQUISITOS DEL SISTEMA

2.1. Requisitos de hardware

Componente	Mínimo	Recomendado
CPU	4 núcleos	8 núcleos
RAM	8 GB	16 GB
Almacenamiento	50 GB SSD	100 GB SSD
Red	100 Mbps	1 Gbps

2.2. Requisitos de software

Componente	Versión	Descripción
Java	21 LTS	JDK para ejecutar los microservicios Spring Boot
Maven	3.8+	Gestión de dependencias y construcción del backend
Node.js	18+	Entorno para Angular y npm
Angular CLI	17+	Herramienta de línea de comandos para Angular
MySQL	5.7	Sistema de gestión de base de datos relacional
Docker	20.10+	(Opcional) Para despliegue con contenedores
Docker Compose	2.0+	(Opcional) Orquestación de contenedores
Git	2.30+	Control de versiones para clonar el repositorio

2.3. Verificación de requisitos

```
# Verificar Java
java -version
# Debe mostrar: openjdk version "21.x.x"

# Verificar Maven
mvn -version
# Debe mostrar: Apache Maven 3.8.x

# Verificar Node.js
```

```
node -v
# Debe mostrar: v18.x.x o superior

# Verificar npm
npm -v
# Debe mostrar: 9.x.x o superior

# Verificar MySQL
mysql --version
# Debe mostrar: mysql Ver 5.7.x

# Verificar Docker (opcional)
docker --version
# Debe mostrar: Docker version 20.10.x o superior
```

3. OBTENCIÓN DEL CÓDIGO FUENTE

3.1. Clonar el repositorio

```
git clone https://github.com/mazgeco/geco-ms.git
cd geco-ms
```

3.2. Estructura del repositorio

```
geco-ms/
 ├── eureka/         # Eureka Discovery
|   ├── gateway/        # API Gateway
|   ├── seguridades/    # Microservicio Seguridades
|   ├── personas/       # Microservicio Personas
|   ├── productos/      # Microservicio Productos
|   ├── proveedores/    # Microservicio Proveedores
|   └── reservas/       # Microservicio Reservas
```

git clone https://github.com/mazgeco/geco-a.git

cd geco-a

```
├── geco-a/          # Aplicación Angular
```

Directorio para el docker:

```
├── docker/
│   ├── docker-compose.yml       # Orquestación con Docker
│   └── Dockerfile.*             # Dockerfiles por servicio
└── docs/
    ├── api/                     # Documentación OpenAPI
    └── deployment/              # Guías de despliegue
```

4. INSTALACIÓN DE LA BASE DE DATOS

4.1. Creación de la base de datos

Conectarse a MySQL y crear las bases de datos para cada microservicio:

```
-- Crear bases de datos
CREATE DATABASE IF NOT EXISTS geco_nombreempresa;

-- Crear usuario y asignar permisos
CREATE USER IF NOT EXISTS 'geco_user'@'%' IDENTIFIED BY 'Geco2026!Secure';
GRANT ALL PRIVILEGES ON geco_nombreempresa.* TO 'geco_user'@'%';
FLUSH PRIVILEGES;
```

4.2. Ejecución de scripts de esquema

```
# Ejecutar scripts de creación de tablas
mysql -u root -p geco_nombreempresa < database/schema/schema.sql
```

4.3. Carga de datos iniciales

Cargar datos maestros
mysql -u root -p geco_nombreempresa < database/initial-data/data.sql

5. CONFIGURACIÓN DE LOS MICROSERVICIOS

5.1. Configuración general

Cada microservicio utiliza un archivo application.yml para su configuración. La estructura base es:

yaml

```
spring:
  application:
    name: [nombre-del-servicio]
  datasource:
    url: jdbc:mysql://localhost:3306/[database_name]
    username: geco_user
    password: Geco2026!Secure
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: false
    properties:
      hibernate:
        dialect: org.hibernate.dialect.MySQL8Dialect
        format_sql: true

server:
  port: [puerto]

jwt:
  secret: ${JWT_SECRET:geco-secret-key-2026}
  expiration: 3600000  # 1 hora
  refresh-expiration: 86400000  # 24 horas
```

5.2. Configuración por servicio

Servicio	Nombre aplicación	Puerto
Gateway	Gateway	8080
Eureka	Eureka	8761
Seguridades	Seguridades	8081
Personas	Personas	8082
Productos	Productos	8083
Proveedores	Proveedores	8084
Reservas	Reservas	8085

5.3. Variables de entorno

Se recomienda utilizar variables de entorno para información sensible:

```
# Variables de entorno requeridas
export JWT_SECRET="geco-secret-key-2026"
export DB_USERNAME="geco_user"
export DB_PASSWORD="Geco2026!Secure"

# Variables opcionales
export SERVER_PORT=8080
export SPRING_PROFILES_ACTIVE=prod
```

6. INSTALACIÓN Y EJECUCIÓN DEL BACKEND

6.1. Instalación de dependencias

Cada microservicio debe ser construido con Maven:

```
# Compilar y empaquetar cada servicio
cd backend/seguridades
mvn clean install -DskipTests
```

```
cd ../personas
mvn clean install -DskipTests

cd ../productos
mvn clean install -DskipTests

cd ../proveedores
mvn clean install -DskipTests

cd ../reservas
mvn clean install -DskipTests

cd ../gateway
mvn clean install –DskipTests

cd ../eureka
mvn clean install -DskipTests
```

6.2. Ejecución de los microservicios

```
# Ejecutar cada servicio en una terminal separada
# Terminal 1 - Gateway
cd backend/gateway
mvn spring-boot:run

# Terminal 2 - Eureka
cd backend/eureka
mvn spring-boot:run

# Terminal 4 - Seguridades
cd backend/seguridades
mvn spring-boot:run
```

```
# Terminal 4 - Personas
cd backend/personas
mvn spring-boot:run

# Terminal 5 - Productos
cd backend/productos
mvn spring-boot:run

# Terminal 6 - Proveedores
cd backend/proveedores
mvn spring-boot:run

# Terminal 7 - Reservas
cd backend/reservas
mvn spring-boot:run
```

6.3. Verificación del estado de los servicios

```
# Verificar que los servicios están corriendo
curl -X GET http://localhost:8080/actuator/health
curl -X GET http://localhost:8761/actuator/health
curl -X GET http://localhost:8081/actuator/health
curl -X GET http://localhost:8082/actuator/health
curl -X GET http://localhost:8083/actuator/health
curl -X GET http://localhost:8084/actuator/health
curl -X GET http://localhost:8085/actuator/health

# Respuesta esperada:
# {"status":"UP"}
```

7. INSTALACIÓN Y EJECUCIÓN DEL FRONTEND

7.1. Instalación de dependencias

```
# Navegar al directorio del frontend
cd frontend/geco-a

# Instalar dependencias Node.js
npm install

# Instalar Angular CLI globalmente (si no está instalado)
npm install -g @angular/cli
```

7.2. Configuración del entorno

El frontend utiliza archivos de entorno para diferentes configuraciones:

typescript

```
// src/environments/environment.ts (desarrollo)
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8080/api/v1',
  authUrl: 'http://localhost:8080/api/v1/auth'
};

// src/environments/environment.prod.ts (producción)
export const environment = {
  production: true,
  apiUrl: 'https://api.gecoerp.com/api/v1',
  authUrl: 'https://api.gecoerp.com/api/v1/auth'
};
```

7.3. Ejecución en modo desarrollo

bash

```
# Iniciar servidor de desarrollo Angular
ng serve

# Acceder a la aplicación en el navegador
# http://localhost:4200
```

7.4. Construcción para producción

```
# Construir la aplicación para producción
ng build --prod

# Los archivos generados se encuentran en dist/geco-a/
```

8. DESPLIEGUE CON DOCKER

8.1. Dockerfiles

Backend (Dockerfile genérico)

dockerfile

```
FROM openjdk:21-slim AS build
WORKDIR /app
COPY mvnw .
COPY .mvn .mvn
COPY pom.xml .
RUN ./mvnw dependency:go-offline -B
COPY src src
RUN ./mvnw package -DskipTests

FROM openjdk:21-slim
WORKDIR /app
```

```
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Frontend (Dockerfile con NGINX)

dockerfile

```
# Fase de construcción
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build --prod

# Fase de producción con NGINX
FROM nginx:alpine
COPY --from=build /app/dist/geco-angular /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

8.2. Docker Compose

yaml

```
version: '3.8'

services:
  mysql:
    image: mysql:8.0
    container_name: geco-mysql
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: geco
    ports:
      - "3306:3306"
    volumes:
      - mysql-data:/var/lib/mysql
      - ./database/schema:/docker-entrypoint-initdb.d
```

```
  networks:
    - geco-network

seguridades:
  build: ./backend/seguridades
  container_name: geco-security
  environment:
    SPRING_PROFILES_ACTIVE: docker
    DB_HOST: mysql
  ports:
    - "8081:8081"
  depends_on:
    - mysql
  networks:
    - geco-network

personas:
  build: ./backend/personas
  container_name: geco-people
  environment:
    SPRING_PROFILES_ACTIVE: docker
    DB_HOST: mysql
  ports:
    - "8082:8082"
  depends_on:
    - mysql
  networks:
    - geco-network

productos:
  build: ./backend/productos
  container_name: geco-products
  environment:
    SPRING_PROFILES_ACTIVE: docker
    DB_HOST: mysql
  ports:
    - "8083:8083"
  depends_on:
    - mysql
  networks:
    - geco-network
```

```
proveedores:
  build: ./backend/proveedores
  container_name: geco-suppliers
  environment:
    SPRING_PROFILES_ACTIVE: docker
    DB_HOST: mysql
  ports:
    - "8084:8084"
  depends_on:
    - mysql
  networks:
    - geco-network

reservas:
  build: ./backend/reservas
  container_name: geco-bookings
  environment:
    SPRING_PROFILES_ACTIVE: docker
    DB_HOST: mysql
  ports:
    - "8085:8085"
  depends_on:
    - mysql
  networks:
    - geco-network

gateway:
  build: ./backend/gateway
  container_name: geco-gateway
  environment:
    SPRING_PROFILES_ACTIVE: docker
  ports:
    - "8080:8080"
  depends_on:
    - securidades
    - personas
    - productos
    - proveedores
    - reservas
  networks:
    - geco-network
```

```
 frontend:
   build: ./frontend/geco-a
   container_name: geco
   ports:
    - "80:80"
   depends_on:
    - gateway
   networks:
    - geco-network

volumes:
 mysql-data:

networks:
 geco-network:
   driver: bridge
```

8.3. Ejecución con Docker Compose

bash

```
# Construir y levantar todos los servicios
docker-compose up -d --build

# Verificar el estado de los contenedores
docker-compose ps

# Ver logs de un servicio específico
docker-compose logs -f gateway-service

# Detener todos los servicios
docker-compose down

# Detener y eliminar volúmenes
docker-compose down -v
```

9. CONFIGURACIÓN DE SEGURIDAD

9.1. Generación de claves JWT

```
# Generar clave secreta para JWT
openssl rand -base64 32

# Resultado: xYz123Abc456Def789Ghi012Jkl345Mno678Pqr901Stu234Vwx
```

9.2. Configuración de HTTPS

Para entornos de producción, es recomendable configurar HTTPS:

```
server:
  port: 8443
  ssl:
    key-store: classpath:keystore.p12
    key-store-password: ${SSL_KEYSTORE_PASSWORD}
    key-store-type: PKCS12
    key-alias: geco
```

Generación del certificado:

```
keytool -genkeypair -alias geco -keyalg RSA -keysize 2048 \
  -storetype PKCS12 -keystore keystore.p12 -validity 3650
```

10. MONITORIZACIÓN Y LOGGING

10.1. Configuración de Logging

```
logging:
  level:
    com.geco: DEBUG
    org.springframework.web: INFO
    org.hibernate: WARN
  file:
```

```
  name: logs/geco.log
  max-size: 10MB
  max-history: 30
pattern:
  console: "%d{yyyy-MM-dd HH:mm:ss} - %msg%n"
  file: "%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n"
```

10.2. Endpoints de monitorización (Actuator)

Endpoint	Descripción
/actuator/health	Estado del servicio
/actuator/info	Información del servicio
/actuator/metrics	Métricas de rendimiento
/actuator/loggers	Configuración de loggers
/actuator/env	Variables de entorno

11. SOLUCIÓN DE PROBLEMAS COMUNES

Problema	Síntoma	Solución
Puerto ocupado	Port 8080 already in use	Cambiar puerto en application.yml o matar proceso: kill -9 $(lsof -t -i:8080)
Error de conexión MySQL	Communications link failure	Verificar que MySQL está corriendo: systemctl status mysql

JWT inválido	JWT signature does not match	Verificar que JWT_SECRET es consistente en todos los servicios
CORS	Access-Control-Allow-Origin	Verificar configuración CORS en Gateway y servicios
Memoria insuficiente	OutOfMemoryError	Aumentar memoria JVM: -Xmx1024m -Xms512m
Dependencias faltantes	ClassNotFoundException	Ejecutar mvn clean install para descargar dependencias

12. CHECKLIST DE DESPLIEGUE

Paso	Verificación	Estado
1	Base de datos MySQL creada y accesible	☐
2	Scripts de esquema y datos iniciales ejecutados	☐
3	Variables de entorno configuradas	☐
4	Cada microservicio compila correctamente	☐
5	Cada microservicio se ejecuta sin errores	☐
6	API Gateway enruta correctamente las peticiones	☐
7	Frontend accede a la API correctamente	☐
8	Autenticación JWT funciona correctamente	☐

9	Todos los módulos están operativos	☐
10	Documentación de API accesible (/swagger-ui.html)	☐

Versión del documento: 1.0

Autores: Naranjo Rodríguez Laura Vanessa, Párraga Castro María Magdalena, Zapata Díaz María Gabriela